

F1 2026 Race Strategy Intelligence Platform

16-Week Execution Plan — Detailed Timeline & Deliverables

Version 1.0 — March 2026

Table of Contents

- 1. [Project Overview & Goals](#)
- 2. [Phase 1 — Data Foundation + Fuel Attribution \(Weeks 1–3\)](#)
- 3. [Phase 2 — Reinforcement Learning Energy Agent \(Weeks 4–7\)](#)
- 4. [Phase 3 — Active Aero Configuration Engine \(Weeks 8–10\)](#)
- 5. [Phase 4 — Overtake Probability & Execution Model \(Weeks 11–14\)](#)
- 6. [Phase 5 — Integration, Dashboard & Documentation \(Weeks 15–16\)](#)
- 7. [Tech Stack Summary](#)
- 8. [Risk Register](#)

Project Overview

This project builds an end-to-end AI/ML platform that models four interconnected aspects of F1 2026 race strategy: sustainable fuel performance attribution, energy deployment optimization, active aerodynamic configuration, and overtake probability prediction. The platform ingests live timing data each race weekend and progressively improves its models as the season unfolds.

The four modules form a pipeline — fuel attribution feeds power curves into the energy agent, which co-optimizes with the aero engine, and the overtake model sits on top as the tactical decision layer. By week 16, the system produces a unified per-lap strategy recommendation for any given race scenario.

Phase	Weeks	Primary ML Technique	Key Output
1 — Fuel Attribution	1–3	Bayesian Regression, SHAP	Fuel performance delta per team
2 — Energy Optimizer	4–7	Reinforcement Learning (PPO/SAC)	Per-lap energy deploy/harvest plan

3 — Aero Engine	8–10	Gaussian Processes, Joint Optimization	Per-sector aero mode schedule
4 — Overtake Model	11–14	XGBoost, LSTM, Game Theory	P(overtake) + attack recommendation
5 — Integration	15–16	Pipeline Orchestration	Live dashboard + final report

Phase 1: Data Foundation + Fuel Attribution — Weeks 1–3

Week 1 — Project Setup & Data Pipeline

Tasks

- Initialize Git repository with standardized project structure: src/, data/, notebooks/, models/, dashboard/, tests/.
- Create Python environment and pin dependencies — FastF1, pandas, NumPy, scikit-learn, PyTorch, XGBoost, GPyTorch, Stable-Baselines3.
- Design and deploy PostgreSQL database schema with tables for: sessions, laps, telemetry snapshots, weather readings, race results, circuit metadata.
- Write data ingestion scripts using the FastF1 / OpenF1 API — pull all available 2026 practice, qualifying, and race sessions. Handle rate limits and caching.
- Build weather data fetcher using OpenWeatherMap or Visual Crossing API — match readings to session timestamps and GPS coordinates.
- Create circuit metadata loader — track length, elevation profile, number of corners, straight lengths, DRS/active-aero zone locations sourced from FIA circuit maps.
- Set up an automated Airflow or Prefect DAG that runs ingestion within 2 hours of each session ending.
- Write data quality check module — detect missing laps, identify outlier times (pit laps, red flags, VSC), flag wet sessions, and log warnings.

Deliverable: A fully automated data pipeline that populates a clean database after every race weekend, with documented schema and data dictionary.

Week 2 — Feature Engineering for Fuel Model

Tasks

- Map each of the 11 teams to their power unit supplier (Ferrari, Mercedes, Honda/RBPT, Audi) and, where publicly known, their fuel partner.
- Engineer lap-level features: fuel-corrected lap time (estimate $\sim 0.03\text{s/kg}$), tire compound, tire age, air temperature, track temperature, humidity, wind speed.
- Build a fuel load estimation sub-model — approximate fuel burn per lap using stint lengths, car weight regulations (768 kg minimum), and known fuel flow limits.
- Create team-level aggregated features: median pace by session type, qualifying delta to pole, race pace standard deviation, and consistency index.
- Engineer interaction features: fuel_supplier $\times$ altitude, fuel_supplier $\times$ ambient_temp (synthetic fuel behavior may vary with conditions).
- Build a track-normalized pace metric — express every lap time as a percentage of the theoretical best at that circuit, enabling cross-track comparisons.
- Write unit tests for all feature engineering functions.

Deliverable: A feature store with 30+ engineered features per lap, documented in a feature catalog spreadsheet.

Week 3 — Fuel Attribution Model (Module A)

Tasks

- Build a Bayesian hierarchical regression model with random effects for team, track, and fuel supplier group. Use PyMC or NumPyro for inference.
- Train the model on all available 2026 data (first 2–3 race weekends). Validate using leave-one-race-out cross-validation.
- Run SHAP analysis on a parallel XGBoost model to decompose lap time into contributions from: fuel/PU, aerodynamic concept, tire management, driver skill, and weather.
- Compute fuel performance delta per team — express as seconds-per-lap advantage/deficit relative to the grid median, with 90% credible intervals.
- Build a causal inference sanity check using difference-in-differences: if any team switches fuel supplier mid-season (unlikely but possible), detect the shift.
- Generate fuel efficiency curves — model how fuel performance varies with fuel load (heavy car vs. light car in a stint).
- Write a Module A API function: get_fuel_delta(team, circuit, conditions) that returns the estimated fuel performance offset.

- Document model assumptions, limitations, and known biases in a technical note.

Deliverable: Module A producing fuel performance deltas with confidence intervals. API function ready for downstream modules.

Milestone 1: End of Week 3 — Data foundation operational. Module A (Fuel Attribution) v1.0 complete and validated on early-season data.

Phase 2: Reinforcement Learning Energy Agent — Weeks 4–7

Week 4 — Simulation Environment Design

Tasks

- Define the Markov Decision Process (MDP) formally. State space: (state_of_charge, fuel_load, tire_compound, tire_age, track_position, gap_ahead, gap_behind, lap_number, sector). Action space: (deploy_level: 0–100%, regen_mode: off/lift/coast, boost_button: yes/no, super_clip: yes/no).
- Build a lap time simulator in Python. Inputs: energy deployed, fuel load, tire grip (degradation curve), aero drag. Output: sector times. Calibrate against real FastF1 data from the first races.
- Model the battery dynamics: charge/discharge rates, max capacity (up from ~4 MJ to ~8.5 MJ in 2026 regs), efficiency losses, thermal limits.
- Integrate Module A fuel curves — the simulator should use the team-specific fuel performance offset when computing power output.
- Model tire degradation as a function of energy deployment (higher deploy → faster through corners → more tire wear). Use simplified Pacejka-style grip curves.
- Wrap the simulator as a Gymnasium-compatible environment with reset(), step(action), render() methods.
- Validate the simulator by replaying known race stints and comparing predicted lap times to actual times (target: within 0.5s/lap).

Deliverable: A calibrated Gymnasium environment that simulates a full F1 2026 race stint with energy, fuel, and tire dynamics.

Week 5 — RL Agent Training (v1)

Tasks

- Implement Proximal Policy Optimization (PPO) using Stable-Baselines3. Configure hyperparameters: learning rate schedule, clip range, GAE lambda, entropy coefficient.

- Design the reward function: $R = -(\text{lap_time}) - \lambda_1 \cdot (\text{energy_violation_penalty}) - \lambda_2 \cdot (\text{ran_out_of_charge_penalty}) + \lambda_3 \cdot (\text{positions_gained_bonus})$. Tune λ values.
- Train initial agent on a single circuit (Melbourne) for 500K+ timesteps. Log training curves in Weights & Biases or TensorBoard.
- Implement curriculum learning: start with no opponents (pure time optimization), then add a pace-car opponent, then a full field of rule-based opponents.
- Benchmark the trained agent against three baselines: (a) constant deploy, (b) linear ramp-down deploy, (c) heuristic strategy from real teams (estimated from timing data).
- Analyze learned policy: generate heatmaps of deploy level vs. SoC vs. lap number. Check for sensible patterns (e.g., higher deploy when gap is small).

Deliverable: PPO agent trained on Melbourne that outperforms all three baselines in simulated race time.

Week 6 — Multi-Circuit Generalization

Tasks

- Extend the simulation environment to support multiple circuits — parametrize by corner count, straight lengths, elevation, and lap count.
- Train a single generalized agent across 3–5 circuits using domain randomization (vary tire deg rates, fuel curves, weather).
- Implement Soft Actor-Critic (SAC) as a second algorithm. Compare sample efficiency and final performance against PPO.
- Add race context to the state: safety car probability model (random events), virtual safety car, and pit stop windows.
- Test zero-shot transfer: evaluate the generalized agent on a circuit it has never trained on. Measure performance gap.
- Implement fine-tuning protocol: given 20 minutes of practice data from a new circuit, fine-tune the generalized agent in under 1000 timesteps.

Deliverable: Generalized RL agent that performs within 2% of circuit-specific agents, with a rapid fine-tuning protocol.

Week 7 — Robustness & Module B API

Tasks

- Stress-test the agent under adversarial conditions: sudden rain, safety car at random laps, incorrect fuel load estimates, battery degradation.
- Implement an ensemble of 5 agents trained with different seeds. Use majority voting or averaged actions for more robust decisions.

- Add explainability: for each lap, log the top 3 state features driving the action choice using integrated gradients or attention weights.
- Build Module B API: `get_energy_plan(circuit, team, conditions, race_state)` → returns per-lap deploy/harvest/boost schedule.
- Write comprehensive unit and integration tests for the environment and agent.
- Document Module B: architecture diagrams, training details, performance benchmarks, and known failure modes.

Deliverable: Module B API producing race-length energy plans. Ensemble agent validated on 5+ circuits.

Milestone 2: End of Week 7 — RL Energy Agent (Module B) v1.0 complete. Integrated with Module A fuel curves. Benchmarked and documented.

Phase 3: Active Aero Configuration Engine — Weeks 8–10

Week 8 — Circuit Geometry Pipeline & Aero Response Model

Tasks

- Build a circuit geometry processor: ingest GPS traces from FastF1, segment tracks into corners and straights, compute curvature profiles, identify aero activation zones.
- Define the aero decision space: for each activation zone, the switchover point from Corner Mode to Straight-Line Mode (and back) is a continuous variable (meters before/after zone boundary).
- Estimate drag and downforce coefficients for each mode using publicly available data, speed trap readings, and corner speed analysis from telemetry.
- Build a Gaussian Process surrogate model that maps (`switchover_point`, `speed_at_switch`, `energy_deployed`) → `sector_time`. Train on simulated data from the environment built in Phase 2.
- Validate the GP model: compare predicted sector times against actual FastF1 sector times. Target $R^2 > 0.85$.
- Create a baseline aero schedule: "switch exactly at zone boundaries" — this is the schedule teams would use without optimization.

Deliverable: Circuit geometry pipeline + GP surrogate model predicting sector times as a function of aero configuration.

Week 9 — Co-Optimization with Energy Agent

Tasks

- Formulate the joint optimization problem: minimize total lap time subject to energy constraints, where both aero switchover points and energy deploy schedule are decision variables.
- Implement alternating optimization: (1) fix energy plan from Module B, optimize aero switchovers via Bayesian Optimization (BoTorch/Optuna), (2) fix aero schedule, re-run RL agent, (3) repeat for 5–10 iterations until convergence.
- Implement a joint approach as comparison: add aero switchover timing as additional actions in the RL agent's action space. Retrain on a single circuit.
- Compare alternating vs. joint optimization — measure lap time improvement and computational cost.
- Run sensitivity analysis: for each switchover point, compute $\partial(\text{lap_time})/\partial(\text{switchover_point})$. Identify the 3 highest-leverage zones per circuit.
- Generate human-readable reports: "Switching to low-drag mode 40m earlier at Turn 5 exit saves 0.08s but costs 0.3 MJ" — this is the kind of output an engineer would want.

Deliverable: Co-optimized energy + aero plans for 3+ circuits, with sensitivity reports per activation zone.

Week 10 — Module C API & Validation

Tasks

- Build Module C API: `get_aero_schedule(circuit, team, conditions, energy_plan)` → returns per-zone switchover timing and expected time gain/loss.
- Validate against real race data: compare model-recommended aero schedules with what teams actually appear to be doing (inferred from speed traces at zone boundaries).
- Test robustness: how does the optimal aero schedule change with wind speed, DRS-train scenarios (multiple cars in a line), and tire compound choice?
- Build visualization: animated circuit map showing the car switching aero modes, overlaid with speed trace and energy level.
- Document Module C: GP model details, optimization approach, convergence behavior, and computational requirements.

Deliverable: Module C API operational. Co-optimization loop converges reliably. Circuit visualizations generated.

Milestone 3: End of Week 10 — Aero Engine (Module C) v1.0 complete. Modules A → B ↔ C pipeline functional end-to-end.

Phase 4: Overtake Probability & Execution Model — Weeks 11–14

Week 11 — Overtake Event Dataset Construction

Tasks

- Define what constitutes an "overtake attempt": car closes to within 1s at a detection line and has Overtake Mode available. Label outcomes as: successful pass, failed attempt, or no attempt.
- Build an automated overtake detector from FastF1 position data — detect when two cars swap positions between consecutive timing sectors.
- Enrich each event with contextual features: gap at detection line, tire compound and age for both cars, SoC estimates for both cars, fuel load delta, lap number, circuit sector, DRS/overtake mode availability.
- Pull track-specific features: overtake zone width (meters of usable track), braking zone length, corner type (hairpin, chicane, long sweeper).
- Compile the dataset from all available 2026 races. Target: 500+ labeled overtake events by this point in the season (approximately 8–9 races in).
- Perform exploratory data analysis: overtake success rate by circuit, by sector, by tire delta, by energy state. Publish an EDA notebook.

Deliverable: Labeled overtake dataset with 40+ features per event and a published EDA notebook.

Week 12 — Overtake Classifier (Static Model)

Tasks

- Train an XGBoost/LightGBM gradient-boosted classifier on the overtake dataset. Target metric: AUC-ROC > 0.78.
- Perform hyperparameter tuning using Optuna with 5-fold cross-validation (stratified by circuit to prevent leakage).
- Run SHAP analysis: identify the top 10 features driving overtake success. Generate summary plots for the final report.
- Build a calibration curve — ensure that when the model says "70% chance of overtake", it succeeds ~70% of the time. Apply Platt scaling if needed.
- Test model on held-out race(s) not seen during training — report precision, recall, F1, and AUC on this true test set.
- Integrate Module A/B/C outputs as features: use the fuel delta from Module A, energy state estimates from Module B, and aero configuration from Module C as additional inputs.

Deliverable: Calibrated overtake probability classifier with AUC > 0.78, integrating outputs from all upstream modules.

Week 13 — Sequence Model & Game-Theoretic Layer

Tasks

- Build an LSTM or Transformer-based sequence model that takes the last 5 laps of battle context (gaps, speeds, energy states) and predicts $P(\text{overtake})$ for the next activation zone. Compare against the static XGBoost model.
- Implement a game-theoretic attacker/defender model: if the attacker boosts, the defender can also boost — model this as a 2-player simultaneous game and compute Nash equilibrium strategies.
- Add counterfactual analysis: "What if the attacker had 0.5 MJ more energy?" — use the classifier to estimate the marginal value of extra energy in overtake scenarios.
- Combine models: use the static classifier as a prior and the sequence model as an update — a Bayesian ensemble that gets more confident as a battle develops.
- Compute expected position gain: for each potential overtake attempt, estimate $E[\text{positions_gained}] \times P(\text{success}) - E[\text{positions_lost}] \times P(\text{failure})$ — a risk-adjusted value.

Deliverable: Ensemble overtake model (static + sequential + game-theoretic) producing risk-adjusted attack recommendations.

Week 14 — Module D API & Backtesting

Tasks

- Build Module D API: `get_overtake_plan(attacker, defender, circuit_sector, race_state)` → returns $P(\text{success})$, recommended boost timing, aero mode, and risk score.
- Backtest the full $A \rightarrow B \leftrightarrow C \rightarrow D$ pipeline on 3 completed races: replay the race lap-by-lap and compare the model's recommended actions to what actually happened.
- Compute a "strategy score": if a team had followed the model's energy, aero, and overtake recommendations, how many positions would they have gained/lost vs. reality?
- Identify failure cases: where did the model give bad advice? Categorize errors (data quality, model bias, unpredictable events) and document fixes for v2.
- Write Module D technical documentation: model architectures, training procedure, calibration results, game-theory formulation, and API reference.

Deliverable: Module D API operational. Full pipeline backtested on 3 races with documented strategy scores.

Milestone 4: End of Week 14 — Overtake Model (Module D) v1.0 complete. All four modules integrated and backtested.

Phase 5: Integration, Dashboard & Documentation — Weeks 15–16

Week 15 — Live Dashboard & End-to-End Pipeline

Tasks

- Build a Streamlit or Plotly Dash dashboard with the following pages:
 - (1) Race Strategy View: select a team and race → see the recommended per-lap energy plan, aero schedule, and overtake opportunities overlaid on a circuit map.
 - (2) Fuel Attribution Dashboard: bar chart of fuel performance delta per team across the season, with confidence bands.
 - (3) Overtake Analyzer: select any battle from any race → see $P(\text{overtake})$ evolution lap-by-lap, with SHAP explanations.
 - (4) Model Performance Tracker: accuracy metrics, calibration curves, and backtest strategy scores updated after each race.
- Wire the full end-to-end pipeline: data ingestion → Module A → Module B ↔ C co-optimization → Module D → dashboard update. Ensure it runs in under 30 minutes per race weekend.
- Add a "live mode" simulation: during a race, manually input current gaps and tire states, and get real-time strategy recommendations.
- Perform load testing and optimize any bottlenecks (the RL fine-tuning step is likely the slowest).
- Deploy the dashboard on a cloud instance (AWS EC2, GCP, or Railway) with a public URL for demo purposes.

Deliverable: Deployed, interactive dashboard backed by the full 4-module pipeline. Accessible via public URL.

Week 16 — Documentation, Report & Presentation

Tasks

- Write the final technical report (20–30 pages) covering: problem statement, literature review of F1 strategy modeling, system architecture, each module's

methodology and results, integrated backtest results, limitations, and future work.

- Prepare a 15-minute presentation deck: problem → approach → architecture diagram → key results (fuel attribution charts, RL training curves, overtake AUC, backtest strategy scores) → live demo → future work.
- Clean and publish the GitHub repository: README with setup instructions, requirements.txt, example notebooks, and module-level documentation.
- Record a 3-minute demo video walking through the dashboard with a real race example.
- Write a retrospective: what worked, what didn't, what would you do differently with 6 more months and real telemetry access?
- Identify v2 enhancements: pit stop strategy integration, weather prediction module, multi-agent RL (model all 22 cars), partnership opportunities with amateur racing teams.

Deliverable: Final report, presentation deck, published GitHub repo, demo video, and v2 roadmap.

Milestone 5 (Final): End of Week 16 — Project complete. All modules integrated, documented, and deployed. Ready for presentation or portfolio.

Tech Stack Summary

Category	Tools
Data Ingestion	FastF1, OpenF1 API, Ergast API, OpenWeatherMap API
Database	PostgreSQL (structured data) + Parquet files (telemetry blobs)
ML / Deep Learning	PyTorch, scikit-learn, XGBoost / LightGBM, GPyTorch
Reinforcement Learning	Gymnasium (custom environment), Stable-Baselines3 (PPO, SAC)

Bayesian Inference	PyMC or NumPyro
Optimization	scipy.optimize, Optuna, BoTorch
Explainability	SHAP, Captum (PyTorch integrated gradients)
Orchestration	Airflow or Prefect
Dashboard	Streamlit or Plotly Dash
Experiment Tracking	Weights & Biases or MLflow
Deployment	Docker + AWS EC2 / GCP / Railway
Version Control	Git + GitHub

Risk Register

Risk 1 — Limited 2026 data early in the season. Mitigation: Use 2022–2025 data for structural model development; switch to 2026 data as it becomes available. The simulator bridges the gap for RL training.

Risk 2 — No access to real telemetry. Mitigation: Rely on FastF1 timing data (sector times, speed traps, gaps) which is publicly available. Supplement with F1 game telemetry exports for high-frequency data.

Risk 3 — Energy/SoC data is not publicly broadcast. Mitigation: Estimate SoC from observable proxies — speed differentials on straights, lift-and-coast patterns, and the new on-car battery indicator lights required by 2026 regs.

Risk 4 — Sim-to-real gap in the RL agent. Mitigation: Domain randomization during training, rapid fine-tuning protocol with real data, and ensemble of agents to reduce variance.

Risk 5 — Overtake dataset too small. Mitigation: Augment with 2022–2025 DRS overtake data (adjusted for new regs), and use synthetic overtake scenarios from the simulator.

Note on scope: This plan is designed for a solo developer or a small team (2–3 people). For a solo build, consider reducing the scope of the game-theoretic layer (Week 13) and the joint RL+aero optimization (Week 9) — the alternating approach alone is sufficient for a strong project.